Reconocimiento de red en entorno Docker

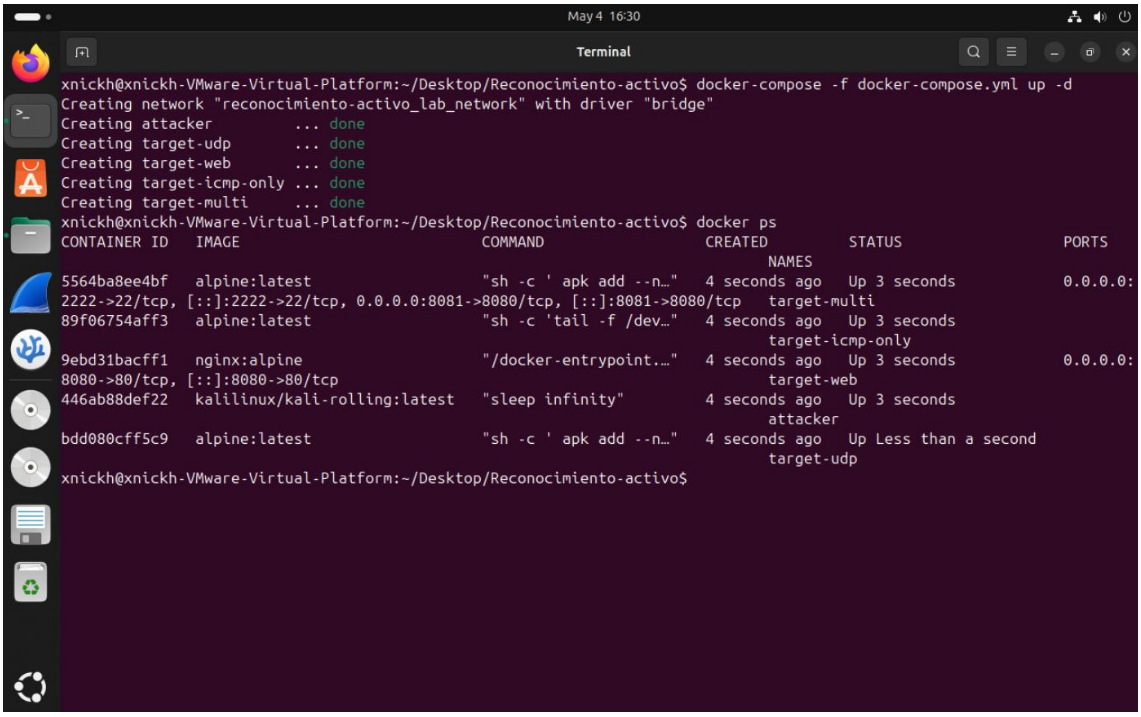
Análisis de tráfico, descubrimiento de hosts y escaneo de puertos con Nmap y tcpdump

Autor: Marcos Pérez García

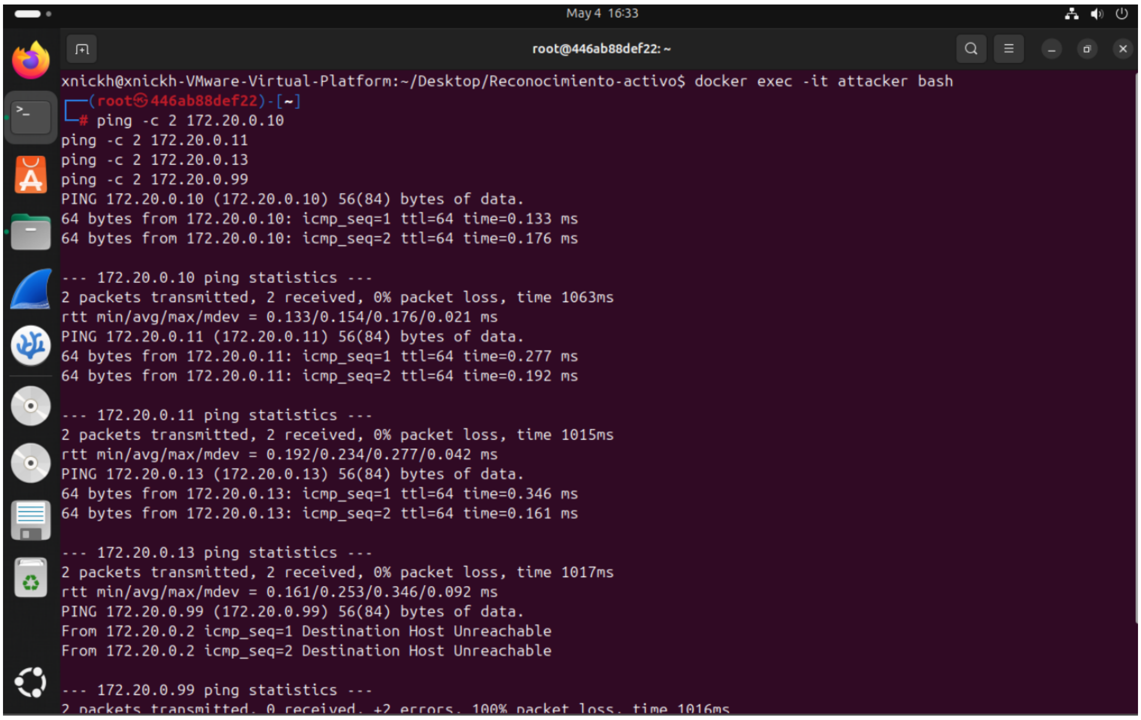
Propósito: Demostración práctica de reconocimiento de red entre contenedores Docker

ÍNDICE

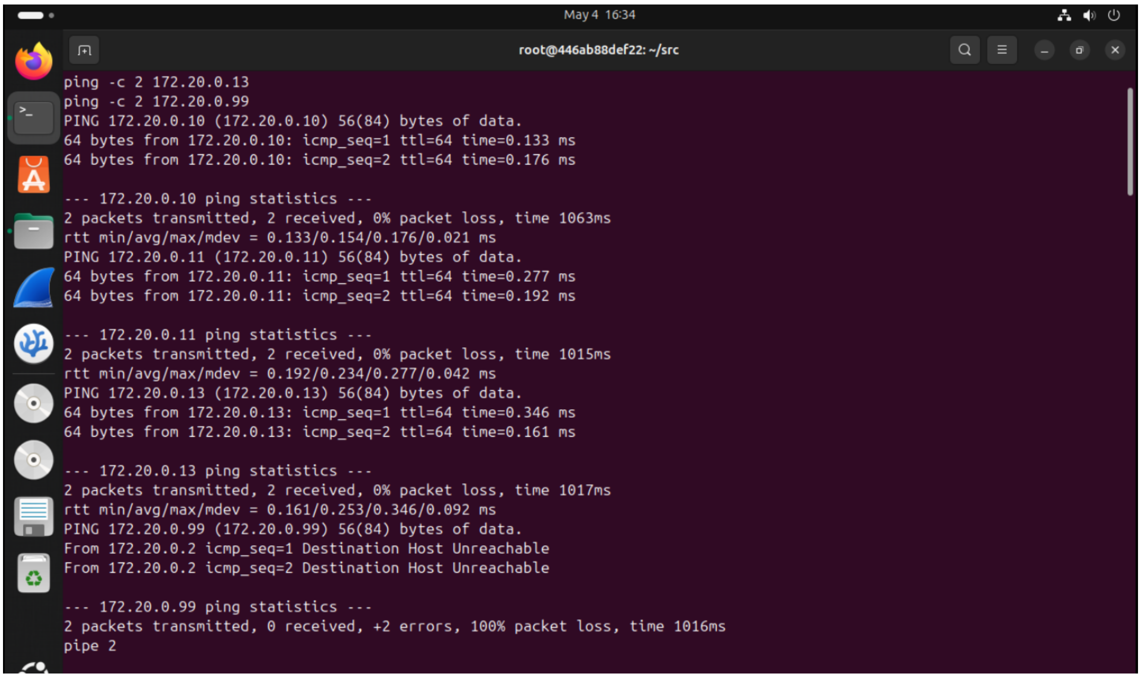
Se inicia el entorno Docker ejecutando docker up para levantar los contenedores definidos en el compose. A continuación, se utiliza docker ps para verificar que los contenedores están en ejecución, mostrando su ID, estado y puertos expuestos.



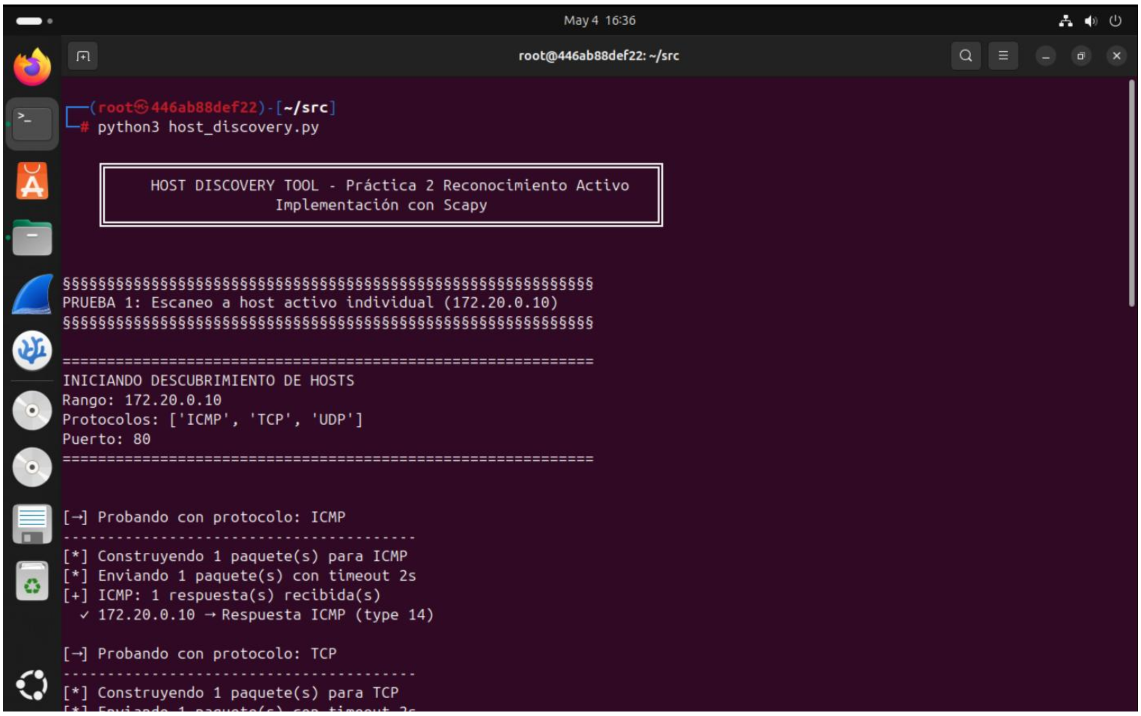
Se ejecuta el contenedor que actuará como atacante. Esto se realiza con docker exec -it <nombre_contenedor> /bin/bash, accediendo así a una terminal interactiva dentro del entorno controlado.



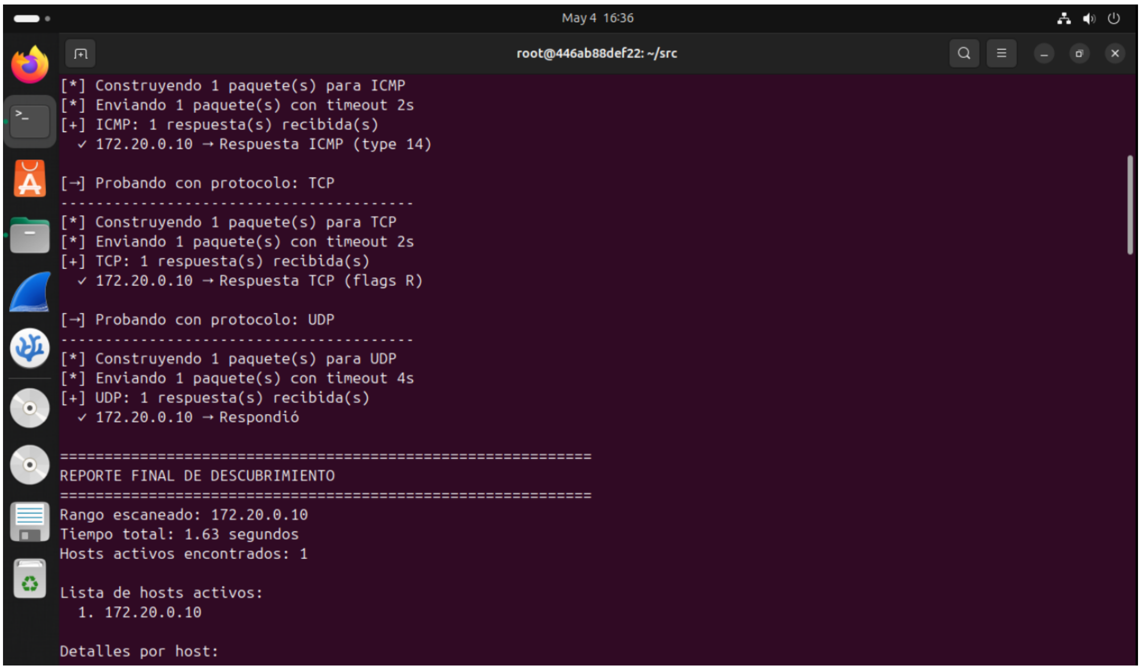
Dentro del contenedor atacante, se realizan pruebas de conectividad usando ping a la IP de la víctima. Esto permite confirmar que ambos contenedores se comunican correctamente a través de la red interna de Docker.

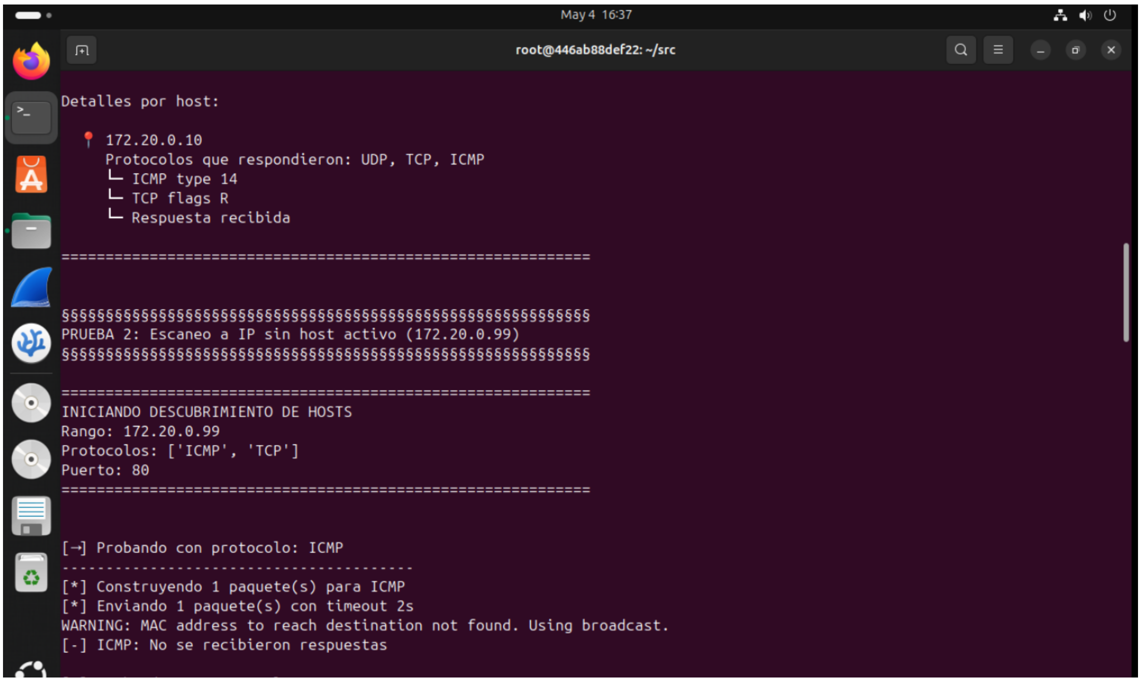


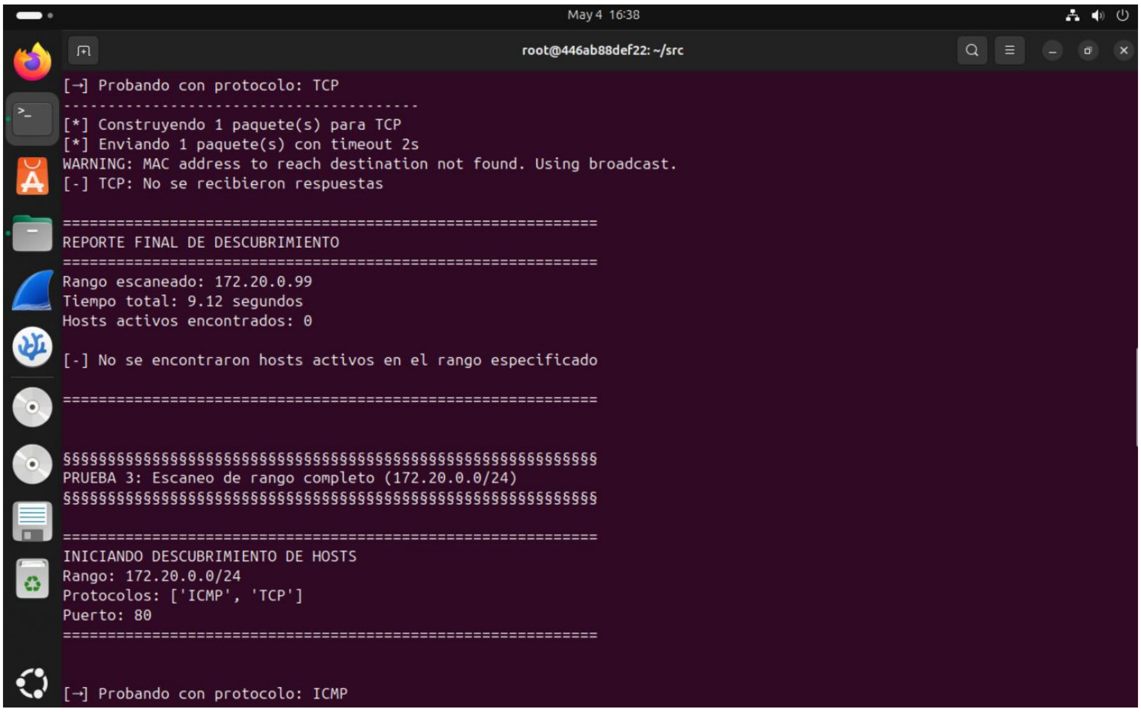
Se ejecuta un script de descubrimiento de hosts (host_discovery) desde el contenedor atacante. Este script envía paquetes ICMP o ARP para identificar qué equipos están activos en la misma subred.

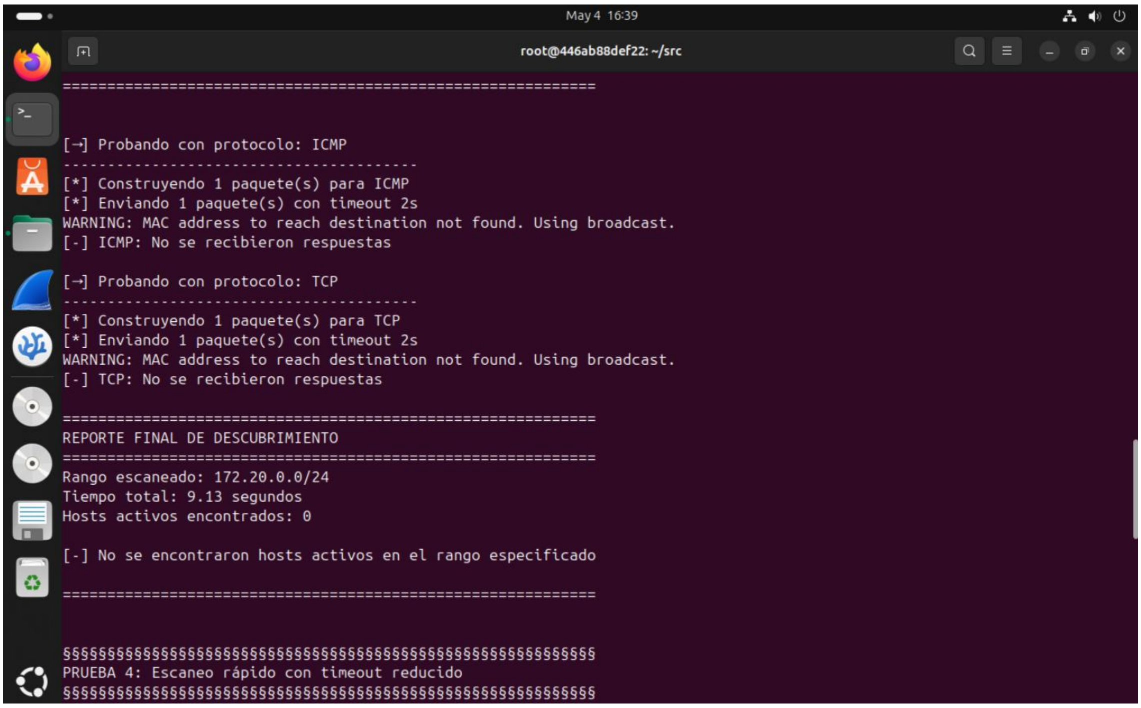


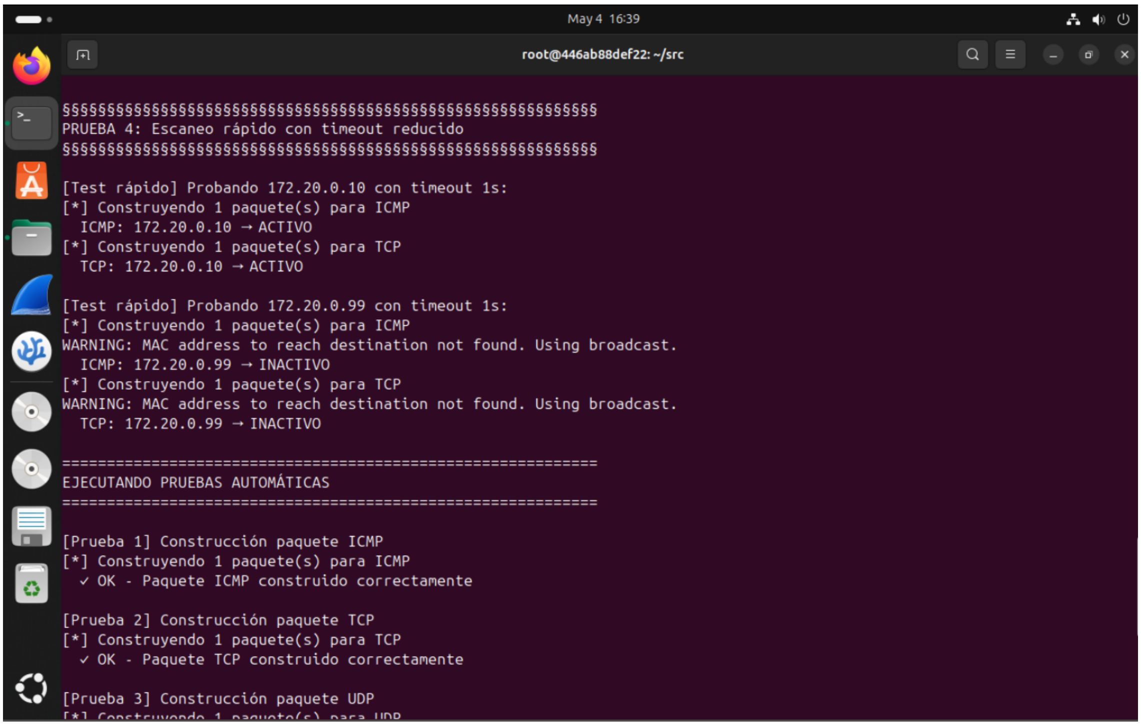
La salida del script muestra las direcciones IP que responden activamente. En este caso, se identifica la IP del contenedor víctima y posiblemente la del gateway de la red Docker.



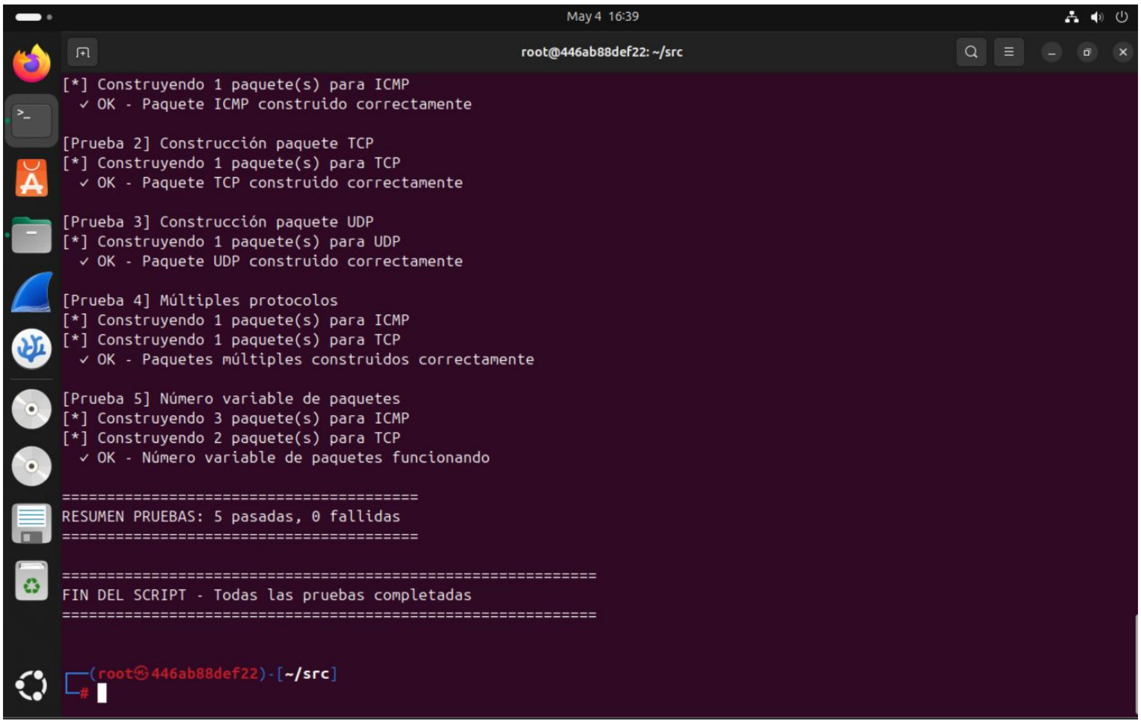



May 4 16:38
root@446ab88def22: ~/src
[→] Probando con protocolo: TCP
----
[*] Construyendo 1 paquete(s) para TCP
[*] Enviando 1 paquete(s) con timeout 2s
WARNING: MAC address to reach destination not found. Using broadcast.
[-] TCP: No se recibieron respuestas
==
REPORTE FINAL DE DESCUBRIMIENTO
==
Rango escaneado: 172.20.0.99
Tiempo total: 9.12 segundos
Hosts activos encontrados: 0
[-] No se encontraron hosts activos en el rango especificado
==
§§
PRUEBA 3: Escaneo de rango completo (172.20.0.0/24)
§§
==
INICIANDO DESCUBRIMIENTO DE HOSTS
Rango: 172.20.0.0/24
Protocolos: ['ICMP', 'TCP']
Puerto: 80
==
[→] Probando con protocolo: ICMP


May 4 16:39
root@446ab88def22: ~/src
==
[→] Probando con protocolo: ICMP
----
[*] Construyendo 1 paquete(s) para ICMP
[*] Enviando 1 paquete(s) con timeout 2s
WARNING: MAC address to reach destination not found. Using broadcast.
[-] ICMP: No se recibieron respuestas
[→] Probando con protocolo: TCP
----
[*] Construyendo 1 paquete(s) para TCP
[*] Enviando 1 paquete(s) con timeout 2s
WARNING: MAC address to reach destination not found. Using broadcast.
[-] TCP: No se recibieron respuestas
==
REPORTE FINAL DE DESCUBRIMIENTO
==
Rango escaneado: 172.20.0.0/24
Tiempo total: 9.13 segundos
Hosts activos encontrados: 0
[-] No se encontraron hosts activos en el rango especificado
==
§§
PRUEBA 4: Escaneo rápido con timeout reducido
§§

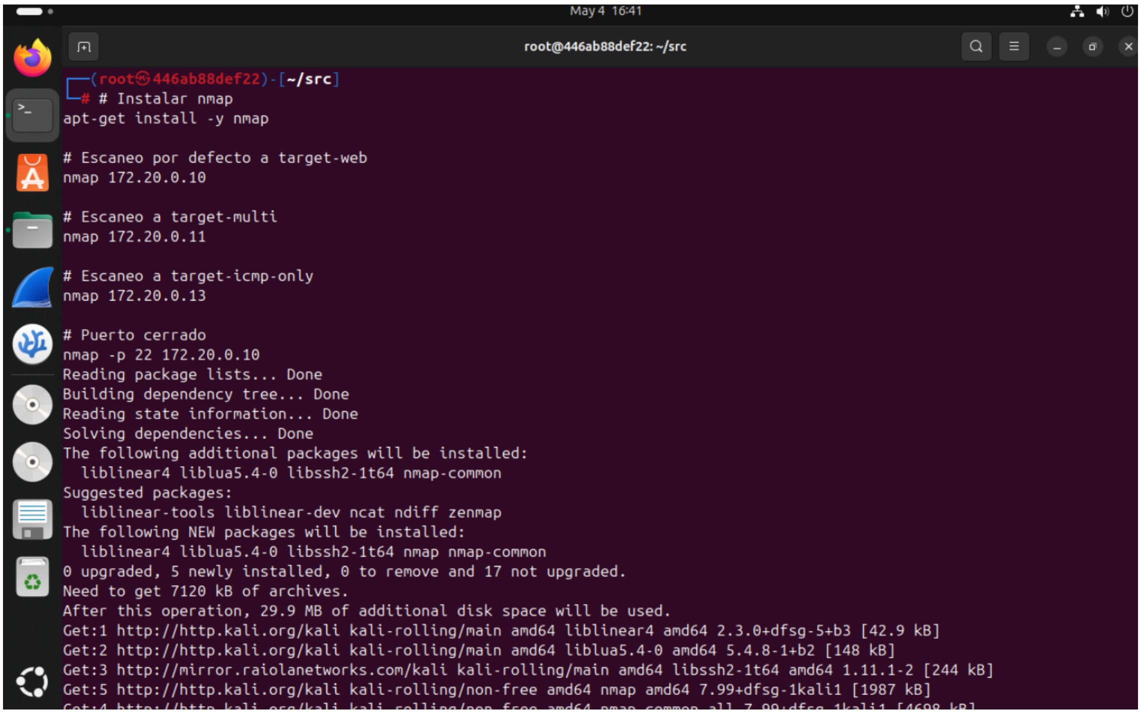


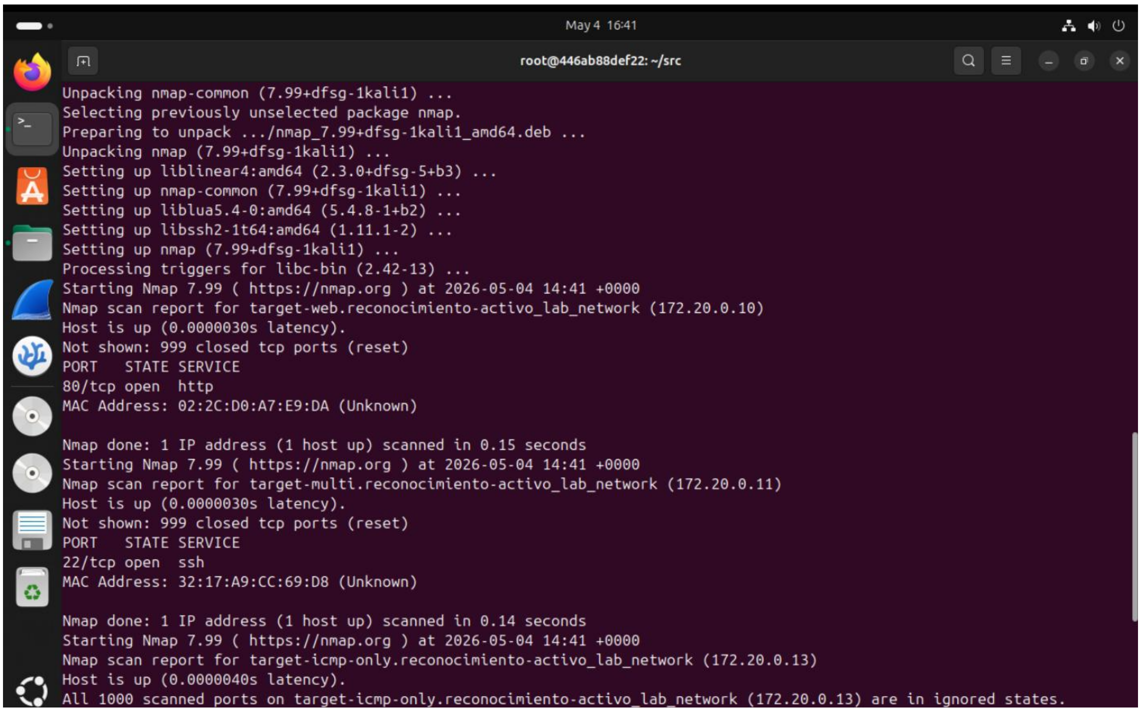
```
§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§
PRUEBA 4: Escaneo rápido con timeout reducido
§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§§

[Test rápido] Probando 172.20.0.10 con timeout 1s:
[*] Construyendo 1 paquete(s) para ICMP
  ICMP: 172.20.0.10 → ACTIVO
[*] Construyendo 1 paquete(s) para TCP
  TCP: 172.20.0.10 → ACTIVO

[Test rápido] Probando 172.20.0.99 con timeout 1s:
[*] Construyendo 1 paquete(s) para ICMP
WARNING: MAC address to reach destination not found. Using broadcast.
  ICMP: 172.20.0.99 → INACTIVO
[*] Construyendo 1 paquete(s) para TCP
WARNING: MAC address to reach destination not found. Using broadcast.
  TCP: 172.20.0.99 → INACTIVO

============================================================
EJECUTANDO PRUEBAS AUTOMÁTICAS
============================================================

[Prueba 1] Construcción paquete ICMP
[*] Construyendo 1 paquete(s) para ICMP
  ✓ OK - Paquete ICMP construido correctamente

[Prueba 2] Construcción paquete TCP
[*] Construyendo 1 paquete(s) para TCP
  ✓ OK - Paquete TCP construido correctamente

[Prueba 3] Construcción paquete UDP
[*] Construyendo 1 paquete(s) para UDP
```

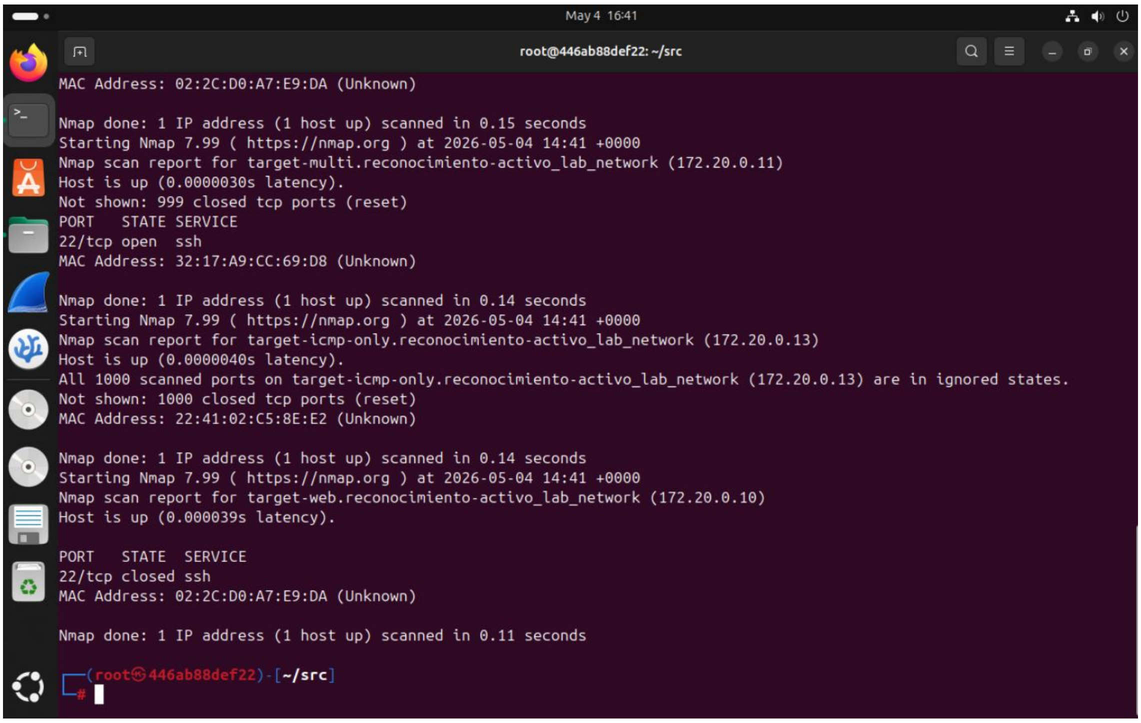


```
[*] Construyendo 1 paquete(s) para ICMP
  ✓ OK - Paquete ICMP construido correctamente

[Prueba 2] Construcción paquete TCP
[*] Construyendo 1 paquete(s) para TCP
  ✓ OK - Paquete TCP construido correctamente

[Prueba 3] Construcción paquete UDP
[*] Construyendo 1 paquete(s) para UDP
  ✓ OK - Paquete UDP construido correctamente

[Prueba 4] Múltiples protocolos
[*] Construyendo 1 paquete(s) para ICMP
[*] Construyendo 1 paquete(s) para TCP
  ✓ OK - Paquetes múltiples construidos correctamente

[Prueba 5] Número variable de paquetes
[*] Construyendo 3 paquete(s) para ICMP
[*] Construyendo 2 paquete(s) para TCP
  ✓ OK - Número variable de paquetes funcionando

========================================
RESUMEN PRUEBAS: 5 pasadas, 0 fallidas
========================================

============================================================
FIN DEL SCRIPT - Todas las pruebas completadas
============================================================

┌──(root㉿446ab88def22)-[~/src]
└─# 
```

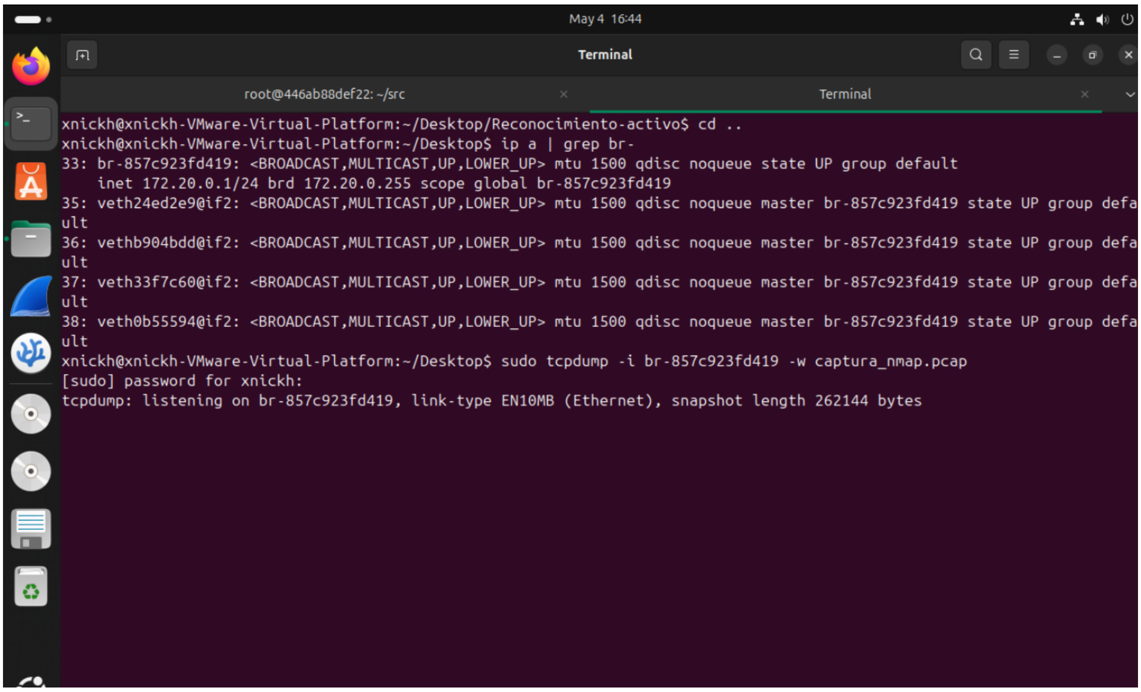
Como Nmap no está disponible por defecto, se instala dentro del contenedor atacante usando apt update && apt install nmap -y. Esto permite realizar escaneos de puertos directamente desde el entorno aislado. Se ejecuta nmap -sS -p- -Pn <IP_victima> para realizar un escaneo sigiloso de todos los puertos. La opción -Pn evita el ping inicial, útil si la víctima bloquea ICMP.



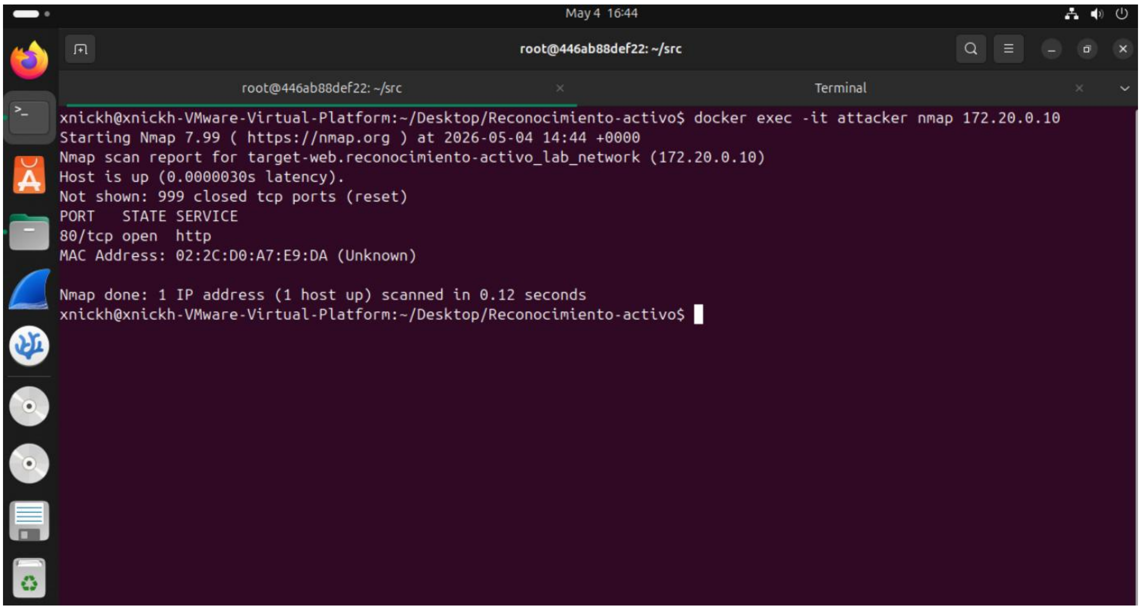




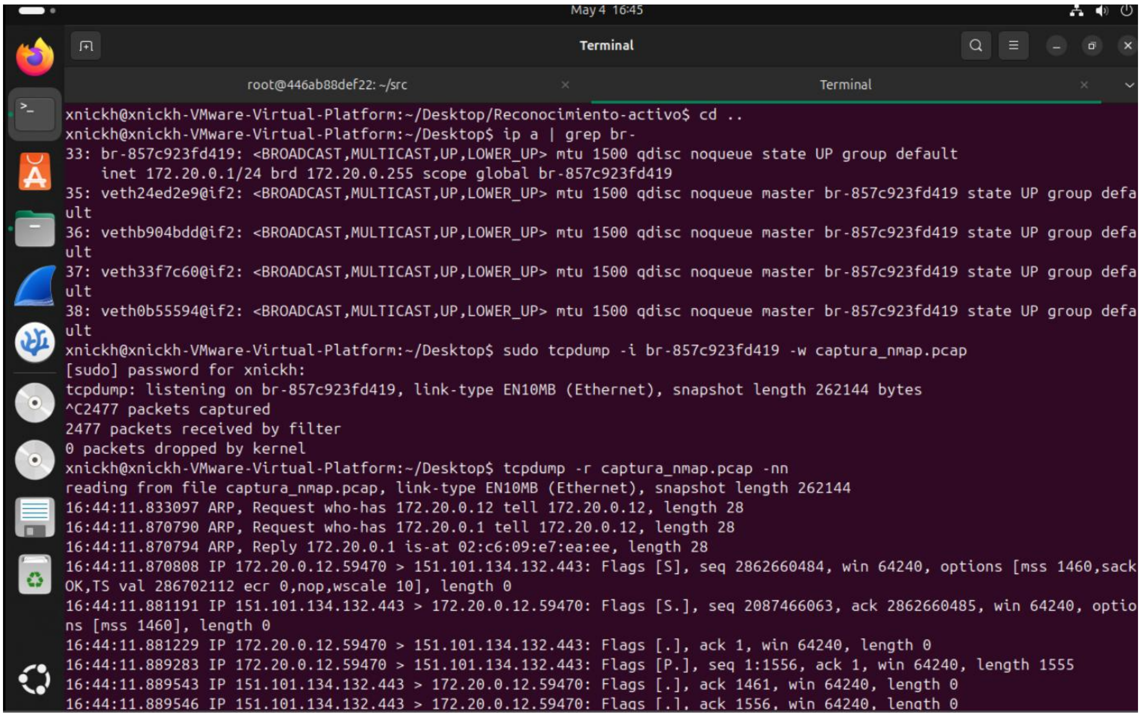
En una segunda terminal, se inicia tcpdump -i eth0 -w captura.pcap para capturar todo el tráfico de red mientras se ejecuta el escaneo. Esto permitirá analizar después los paquetes enviados y recibidos.

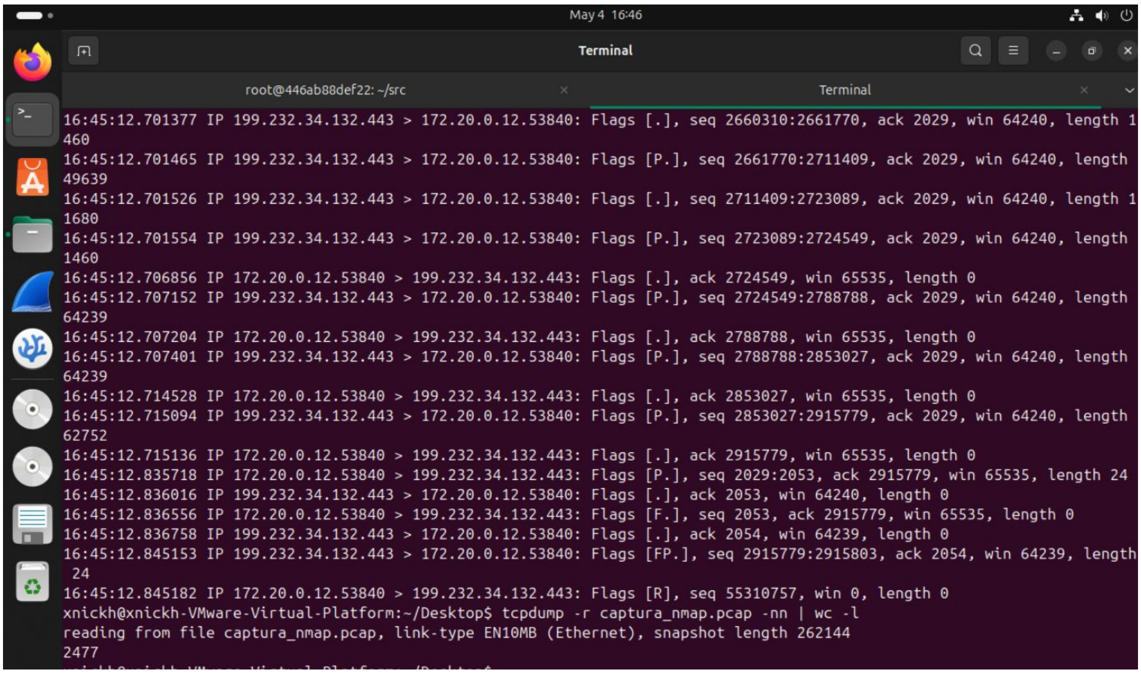


Mientras tcpdump está en ejecución, se lanza el escaneo de Nmap desde el contenedor atacante. Toda la actividad queda registrada en el archivo PCAP para su posterior análisis

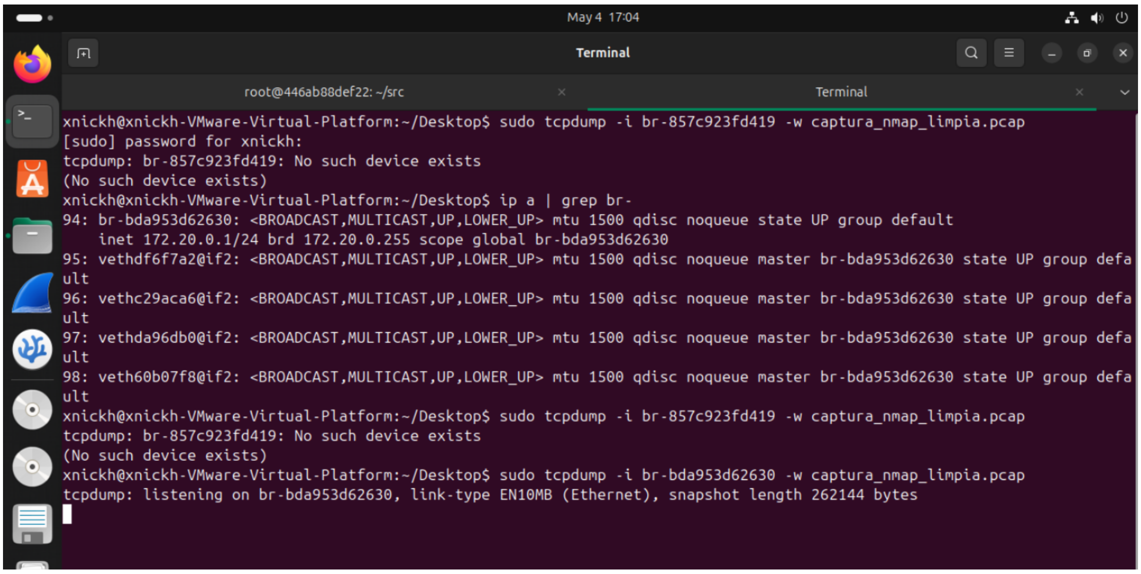


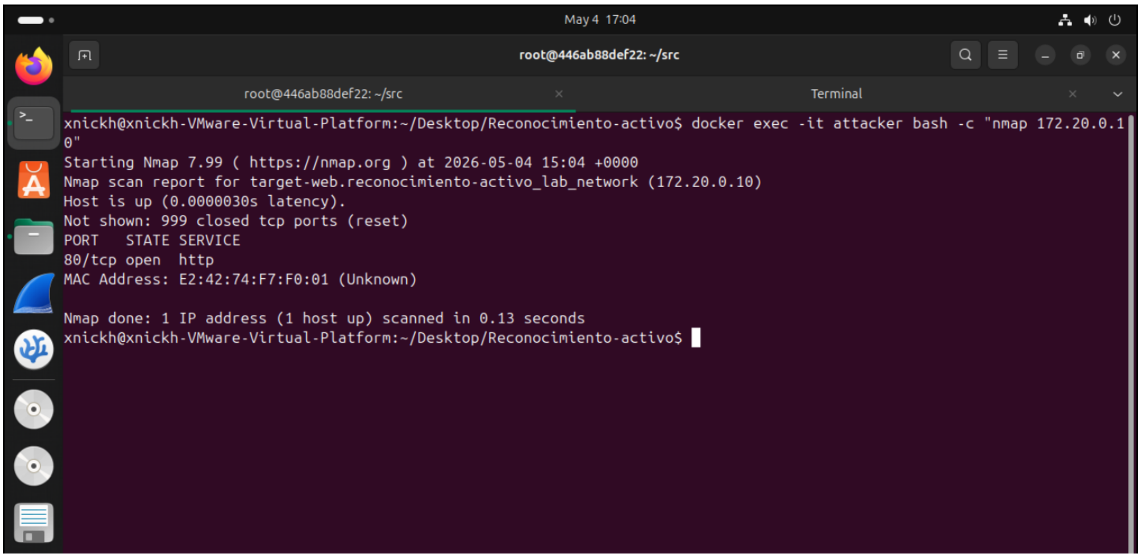
Una vez terminado el escaneo, se detiene tcpdump y se visualiza la captura con tcpdump -r captura.pcap -n. Se observan paquetes TCP SYN, SYN-ACK, RST y posiblemente ICMP, según el tipo de escaneo.

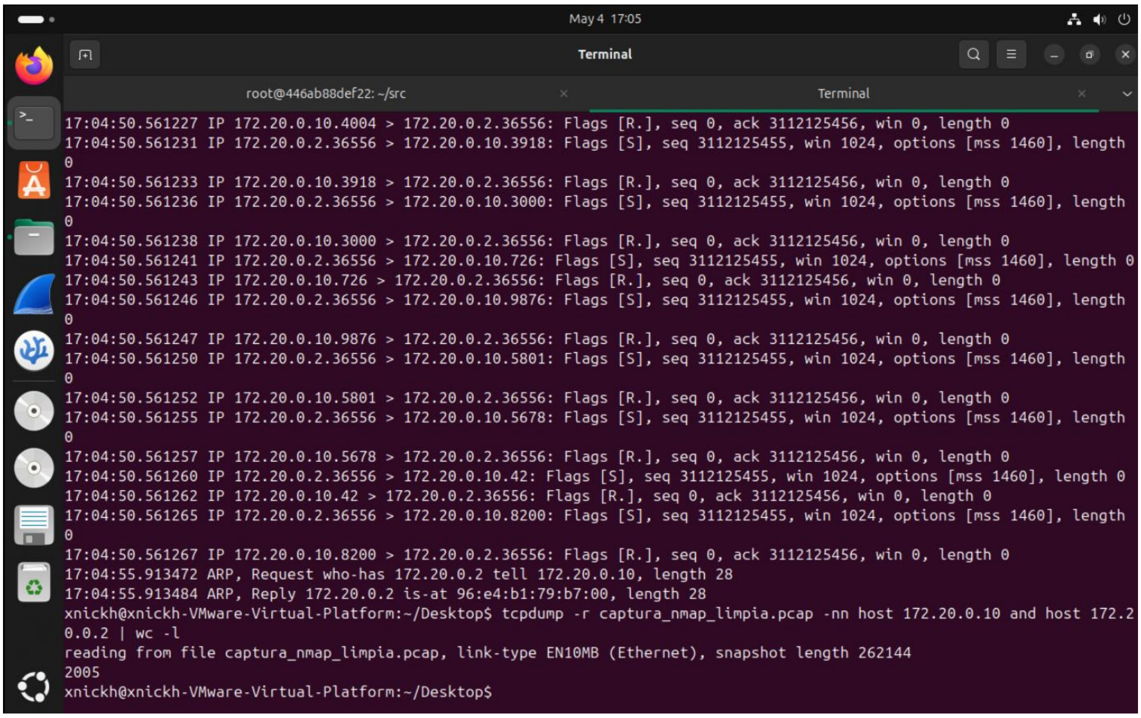




Se repite el procedimiento, pero esta vez cambiando el tipo de escaneo (por ejemplo, -sT para conexión completa) o modificando los puertos a escanear, con el fin de comparar el tráfico generado.







Se inicia nuevamente tcpdump y se ejecuta el segundo escaneo. Es importante limpiar las capturas anteriores o usar un nombre diferente para no sobrescribir los datos.

Al revisar la segunda captura, se pueden notar diferencias en los flags TCP, reintentos, respuestas ICMP tipo 3 (puerto inaccesible) o patrones de conexión más intrusivos según el método usado.

Finalmente, se han completado todas las fases del reconocimiento:

1. Verificación de conectividad.
2. Descubrimiento de hosts.
3. Escaneo de puertos con Nmap.
4. Captura y análisis de tráfico con tcpdump.

Los resultados permiten identificar servicios potencialmente vulnerables en la víctima.